```
//You need to import sqlite3 header file for implementing sqlite Database demo.
import SQLite3

//structure for table fields.
struct Employee {
   var id: Int
   var name: String
   var age: Int
}

//we are using tableView controller thats why we are inheriting UITableViewDelegate,
//UITableViewDatasource overhere
class ViewController: UIViewController, UITableViewDelegate, UITableViewDataSource {

//outlet for inputs and table view
@IBOutlet weak var AgeInput: UITextField!
@IBOutlet weak var nameInput: UITextField!
@IBOutlet weak var rollInput: UITextField!
@IBOutlet weak var tblView: UITableView!

//variable for database path or lets say database name, whatever you want to give as database
name
let dbPath: String = "myDb.sqlite"

//OpaquePointer is a special Swift pointer type used to represent a C pointer when Swift does
not know its internal structure. And here it is use as database pointer.
var db: OpaquePointer?

//variable for employee list array which variable is use for store data in this code.
var empList = [Employee]()

override func viewDidLoad() {
      super.viewDidLoad()

//call function to open database
      db = openDatabase()
//call function to create table.
      CreateTable()
//call function to fetch data from table
      loadData()
}


 func loadData () {
```

```
//fetch data from table in empList variable. In read() function, select query is mention.
    empList = read()
// reload table view for getting updated data in table.
    tblView.reloadData()
}

//function for getting the number of rows in table (we did in tableview demo)
func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
    return empList.count
}



//function to fill table (we did in tableView demo)
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) -> UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "cell", for: indexPath)
    let emp = empList[indexPath.row]
    cell.textLabel?.text = emp.name
    cell.detailTextLabel?.text = "ID: \(emp.id) | Age: \(emp.age)"


    return cell
  }




//function to perform when click on insert button for inserting data
@IBAction func handleBtnAction(_ sender: UIButton)
{
    if (sender.titleLabel?.text == "Insert") {
        // same code as we did in dynamic table view demo
        insert(id: Int(rollInput.text!) ?? 0, name: nameInput.text!, age: Int(AgeInput.text!) ?? 0)
        let emp = Employee(id: Int(rollInput.text!) ?? 0, name: nameInput.text!, age:
        Int(AgeInput.text!) ?? 0)
        rollInput.text = ""
        nameInput.text = ""
        AgeInput.text = ""
        empList.append(emp)
         tblView.reloadData()
    } else {
        // update function calloing if button text is update and reload tableview data after
        //update data
        update()
        tblView.reloadData()
    }
}
```

```
// MARK: - Open Database
func openDatabase() -> OpaquePointer? {
    //getting the database file url where our created database is stored.
    let fileURL = try! FileManager.default
      .url(for: .documentDirectory,
          in: .userDomainMask,
          appropriateFor: nil,
          create: false)
      .appendingPathComponent(dbPath)


    //first set the db pointer as nil
    var db: OpaquePointer? = nil


//check whether database file is available on that specific path url or not. If available then it
return database file otherwise print message “Unable to open database”.
    if sqlite3_open(fileURL.path, &db) != SQLITE_OK {
      print("Unable to open database")
      return nil
    } else {
      print("Database opened at \(fileURL.path)")
      return db
    }
  }



  // MARK: - Create Table
  func createTable() {
    //declare the create table query
    let createTableString = """
      CREATE TABLE IF NOT EXISTS person(
      Id INTEGER PRIMARY KEY,
      name TEXT,
      age INTEGER);
      """


    var createTableStatement: OpaquePointer? = nil


     // Prepare (compile) the SQL statement
    if sqlite3_prepare_v2(db, createTableString, -1, &createTableStatement, nil) == SQLITE_OK {


          // If preparation is successful, execute the statement
          if sqlite3_step(createTableStatement) == SQLITE_DONE {
                print("person table created.")
          } else {
```

```
            print("person table could not be created.")
        }

        } else {
            print("CREATE TABLE statement could not be prepared.")
    }
    // Free the memory used by prepared statement
    sqlite3_finalize(createTableStatement)
}



// MARK: - Insert Data
func insert(id: Int, name: String, age: Int) {
 // SQL query with placeholders (?) for values
// ? are used to prevent SQL injection and allow safe binding
let insertStatementString = "INSERT INTO person (Id, name, age) VALUES (?, ?, ?);"


    var insertStatement: OpaquePointer? = nil
    // Prepare (compile) the SQL statement
   // If preparation is successful, it returns SQLITE_OK
    if sqlite3_prepare_v2(db, insertStatementString, -1, &insertStatement, nil) == SQLITE_OK {
        // Bind integer value to first placeholder (position starts from 1)
        // 1 → first ? → Id column
        sqlite3_bind_int(insertStatement, 1, Int32(id))


        // Bind text value to second placeholder
        // 2 → second ? → name column
        sqlite3_bind_text(insertStatement, 2, (name as NSString).utf8String, -1, nil)


        // Bind integer value to third placeholder
        // 3 → third ? → age column
        sqlite3_bind_int(insertStatement, 3, Int32(age))


        // Execute the prepared statement
        // If execution successful → returns SQLITE_DONE
        if sqlite3_step(insertStatement) == SQLITE_DONE {
            print("Successfully inserted row.")
        } else {
            print("Could not insert row.")
        }
    } else {
       print("INSERT statement could not be prepared.")
    }
    // Free memory used by prepared statement
   // Very important to avoid memory leak
```

```
    sqlite3_finalize(insertStatement)
}



// MARK: - Read Data
func read() -> [Employee] {
    // SQL query to fetch all rows from person table
    let queryStatementString = "SELECT * FROM person;"
    var queryStatement: OpaquePointer? = nil

     // Empty array to store fetched Employee objects
    var employees: [Employee] = []

      // Prepare (compile) the SELECT query
    // If successful → returns SQLITE_OK
    if sqlite3_prepare_v2(db, queryStatementString, -1, &queryStatement, nil) == SQLITE_OK {
        // Execute the query row by row
       // SQLITE_ROW means one row of data is available
        while sqlite3_step(queryStatement) == SQLITE_ROW {


            // Read integer value from column index 0 (Id column)
            let id = sqlite3_column_int(queryStatement, 0)
            // Read text value from column index 1 (name column)
            // Convert C string to Swift String
           let name = String(cString: sqlite3_column_text(queryStatement, 1))


            // Read integer value from column index 2 (age column)
           let age = sqlite3_column_int(queryStatement, 2)


            // Create Employee object and add to array
           employees.append(Employee(id: Int(id),
                                     name: name,
                                     age: Int(age)))
        }


    } else {
        print("SELECT statement could not be prepared")
    }


     // Release memory used by prepared statement
    sqlite3_finalize(queryStatement)


   // Return all fetched employee records
    return employees
```

```
    }



    // MARK: - Delete Data
    func deleteByID(id: Int) {
        // SQL query to delete a row where Id matches the given value
        // ? is a placeholder for safe value binding
        let deleteStatementString = "DELETE FROM person WHERE Id = ?;"
        var deleteStatement: OpaquePointer? = nil


        // Prepare (compile) the DELETE SQL statement
        // If preparation is successful → returns SQLITE_OK
        if sqlite3_prepare_v2(db, deleteStatementString, -1, &deleteStatement, nil) == SQLITE_OK {
                // Bind the Id value to the first placeholder (?)
                // Index starts from 1 in SQLite binding
                sqlite3_bind_int(deleteStatement, 1, Int32(id))


                // Execute the DELETE statement
                // If successful → returns SQLITE_DONE
                if sqlite3_step(deleteStatement) == SQLITE_DONE {
                        print("Successfully deleted row.")
                } else {
                        print("Could not delete row.")
                }


        } else {
            print("DELETE statement could not be prepared.")
        }


        // Free the memory used by prepared statement
        // Important to prevent memory leak
        sqlite3_finalize(deleteStatement)
    }
}


// MARK: -  Table view method (when we click on particular row that data must be fill in input field)
func tableView(_ tableView: UITableView, didSelectRowAt indexPath: IndexPath) {
        let selectedData = empList[indexPath.row]
        rollInput.text = "\(selectedData.id)"
        nameInput.text = selectedData.name
        AgeInput.text = "\(selectedData.age)"
        guard let rInput = rollInput.text,!rInput.isEmpty,
            let nInput = nameInput.text, !nInput.isEmpty,
            let aInput = AgeInput.text, !aInput.isEmpty else {return}
```

```
    // Set button text as “Update”
    btn.setTitle("Update", for: .normal)
  }


// MARK: - Update Data
  func update() {
        // Step 1: Get values from textfields safely using guard
        // If any field is empty → stop execution
        guard let rollText = rollInput.text,
            let name = nameInput.text,
            let ageText = AgeInput.text,
            !rollText.isEmpty,
            !name.isEmpty,
            !ageText.isEmpty else {
                print("All fields required")
                return
                }
        // Step 2 : Create SQL UPDATE query string
        // This will update name and age where id matches roll number
        let updateStatementString = """
            UPDATE person SET name = "\(name)", age = \(ageText) WHERE id = \(rollText);
        """
        // Step 3: Execute the query using sqlite3_exec
        if (sqlite3_exec(db, updateStatementString,nil, nil, nil) == SQLITE_OK) {
            print("Student updated successfully")

            // Step 4: Update local array (empList) using map()
            // We must update our data source to reflect changes in UITableView
            let newArray = empList.map{p in

            // Check if current object's id matches updated id
            if (p.id == Int(rollText)) {
                print(p)
                // Return updated Employee object
                return Employee(id: Int(rollText)!, name: name, age: Int(ageText)!)
            } else {
                // If not matching → return original object
                return p
            }}

            // Step 5: Replace old array with updated array
            empList = newArray

            // Step 6: Reload UITableView to reflect changes on screen
            tblView.reloadData()
        } else {
            // If SQL execution fails
            print("Update failed")
        }
```

}

The Core SQLite functions

Lets first start with a list of the most used SQLite functions and describe their purpose:

- sqlite3_open(): This function creates and opens an empty database with the specified filename argument. If the database already exists it will only open the database. Upon return the second argument will contain a handle to the database instance.
- sqlite3_close(): This function should be used to close a previously opened SQLite database connection. It will free all system resources associated with the database connection.
- sqlite3_prepare_v2(): To execute an SQL statement it first needs to be compiled into byte-code and that is exactly what this function is doing. It basically transforms an SQL statement written in a string to an executable piece of code.
- sqlite3_step(): Calling this function will execute a previously prepared SQL statement.
- sqlite3_finalize(): This function deletes a previously prepared SQL statement from memory.
- sqlite3_exec(): Combines the functionality of sqlite3 sqlite3_prepare_v2(), sqlite3_step() and sqlite3_finalize() into a single function call.
- sqlite3_column_<type>(): This routine returns information about a single column of the current result row of a query. Typical values for <type> are text and int. It is important to note that the column indexes are zero based (start with zero).
- sqlite3_bind_<type>(): This function is use for binding value(add or update or delete value). Typical values for <type> are text and int (any datatype used in your database). It is important to note that the bind indexes are one based (start with one).

A full list of functions can be found on the SQLite website.